# Contents

# 1 XML (Extra slides)

## 1.1 Introduction

### 1.1.1 About your instructor

- Jan Ypma
  - Independent software architect
  - Java, Scala, Groovy, C++, Rust, Lisp
  - Contributor to various open source projects
  - Fan of functional programming and distributed systems
  - Agile coach and fan of Domain-Driven Design
  - `jan@ypmania.net`, `https://linkedin.com/in/jypma`

### 1.1.2 Schedule, day 1

| Time | Duration | Activity | Slides |
|---|---|---|---|
| 09:00 | 00:30 | Welcome, Outline/Agenda, Expectations | 1-10 |
| 09:30 | 00:20 | Round table introductions | 11 |
| 09:50 | 00:30 | Exercise 1,2,3: Installation, git, and repo | 12-18 |
| 10:20 | 0:05 | If needed: Command-Line Tooling | |
| 10:25 | 00:25 | Introduktion til XML | 19-22, 23-24 |
| 10:50 | 0:10 | Intermezzo: XML history | |
| 11:00 | 0:15 | Share your own XML and XSD | |
| 11:15 | 00:15 | Exercise 4: Follow data structure | 25 |
| 11:30 | 00:30 | Exercise 5: Draw, and create some XML | 26 |
| 12:00 | 00:20 | Exercise 5B: JSON to XML | 27 |
| 12:20 | 00:30 | Lunch | |
| 12:50 | 00:30 | Intermezzo: Bytes and encodings | |
| 13:20 | 00:30 | XML building blocks | 28-35 |
| 13:50 | 0:30 | Bonus exercise (if desired): Git and github | |
| 14:20 | 00:30 | Exercise 6: Create well-formed XML | 36 |
| 14:50 | 00:30 | Namespaces | 37-45 |
| 15:20 | 00:15 | XML Demo: UBL invoice (XML only) | |
| 15:35 | 00:20 | Exercise 7: Namespaces for TV Guide | |
| 15:55 | 00:15 | Exercise Bonus-1: Docbook | |
| 16:10 | 00:30 | Wrap-up, reserved time for extra subjects | |

### 1.1.3 Schedule, day 2

| Time | Duration | Activity | Slides |
|---|---|---|---|
| 09:00 | 00:10 | Welcome, Outline/Agenda | |
| 09:10 | 0:30 | Intermezzo: Document types (DTD) | |
| 09:40 | 0:15 | Introduction to XPath | 6,9-13 |
| 09:55 | 0:15 | XPath demo | |
| 10:10 | 0:10 | XSD Introduction | 3,4 |
| 10:20 | 0:20 | About schemas and types | 6-10 |
| 10:40 | 0:10 | Simple types | 12-15 |
| 10:50 | 0:30 | Exercise XSD-1 | |
| 11:20 | 0:30 | Complex types | 18-30 |
| 11:50 | 0:30 | Option: Describe UBL | |
| 12:20 | 0:30 | Exercise XSD-2 (incl. modeling / ev.st.) | |
| 12:50 | 00:30 | Lunch | |
| 13:20 | 0:20 | Advanced choices | 32-35 |
| 13:40 | 0:20 | Modularization | 37-40 |
| 14:00 | 0:30 | Intermezzo: Data modeling techniques | |
| 14:30 | 0:20 | XML Demo: UBL invoice XSD structure | |
| 14:50 | | Intermezzo: Welcome to the world of JSON | |
| 14:50 | 0:10 | Best practices | 42-43 |
| 15:00 | 00:60 | Wrap-up, reserved time for extra subjects | |

## 1.2 XML history

Obligatory XKCD

- 1986: SGML: Standard Generalized Markup Language (first introduction of `<tags></tags>`, with some wild other syntaxes)
- 1990: Birth of HTML
- 1996: XML 1.0, first spec (from *text* to *data*)
- 1999: First draft for *namespaces* in XML
- 2004: XML 1.1
- 2004: XSD 1.1 (first usable definition, including namespaces)

## 1.3 Command-line tooling

### 1.3.1 Run an XPath expression

Using `xmllint` (in `libxml2`)

```
xmllint --xpath "//Vej" ../LB2744/examples/xml/02/order.xml | wc -l
```

bash

### 1.3.2 Validate XML with XSD

Using `xmllint` (in `libxml2`)

```
cd ../LB2744/examples/xsd/07
xmllint --noout --schema PersonType.xsd Person.xml 2>&1
```

### 1.3.3 Execute XSLT

Using `xsltproc` (in `libxslt`)

```
cd ../LB2744/examples/xsl/02
xsltproc --output out.html xml2html.xsl zoo.xml
```

## 1.4 Technology

**Online**

- Excalidraw

**Windows**

- Notepad++

**Multi-platform**

- VS Code
- IntelliJ

**Author's choice**

- Emacs

# 2 Bytes and encodings

## 2.1 Character encodings

### 2.1.1 History

- Early computer systems: 7 bits (last bit not transmitted, or used for parity)
- 1961: ASCII (7-bit)
- *Nøglen er sær*
- 1978: Code pages (DOS 3.3)
- 1985: ISO-8859-1
- 1991: First unicode (16-bit)
- 1996: Unicode extended to 21-bit (about 1.000.000 code points)
- 1998: UTF-8

## 2.2 Whitespace

### 2.2.1 Line endings

- New line in text documents (including XML):
  - Windows: `\r\n` (carriage return, line feed) or in bytes: (13, 10)
  - Mac, Linux: `\n` (line feed) or in bytes: (10)
  - (old) AS400 (IBM): `0x85` (NEL, 133)
- This brings issues when collaborating
  - Both for humans and systems alike

### 2.2.2 Whitespace

- Are the following documents equal:

```
<order><description>An electrical connector</description></order>
```

and

```
<order>
  <description>
    An electrical connector
  </description>
</order>
```

### 2.2.3 Whitespace

- Are the following documents equal:

```
<order>
  <description>
    A connector looking like this:
    |-------|
    |   o   |
    | o   o |
    |-------|
  </description>
</order>
```

and

```
<order>
  <description>
    A connector looking like this:    |-------|    |   o   |    | o   o |    |-------|
  </description>
</order>
```

## 2.3 Encoding demos

### 2.3.1 Let's look at some UTF-8

```
cat ../LB2744/examples/xml/02/order.xml | xxd
```

### 2.3.2 Let's look at some ISO-8859-1

Can you guess which line endings are used here?

```
cat ../LB2744/examples/xml/nationalbanken/CurrencyRatesXML.xml | xxd
```

## 2.4 Conclusions

Communicate expectations. Saying "XML" isn't enough.

- How are we to interpret white space?
- What line endings do we use?
- What character encoding do we use?
- Is there an XSD, DTD, or otherwise?

# 3 Recap day 1

- Who can tell us something about:

  Attribute Carriage Return Commit Element Namespace Prefix Push `standalone` UTF-8

# 4 DTDs

## 4.1 Introduction

*Document Type Definition (DTD)*

- What tags can I expect in this document?
- Are these documents of the same *type*?

## 4.2 Functionality

- Define the root element
- Define the expected content, and order, of elements
- Define attributes that are allowed on elements
- Define entities (expanding snippets of text)

## 4.3 Defining the document type in XML

### 4.3.1 Define as internal DTD

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE zoo [
<!ELEMENT zoo (animal)+>
<!ELEMENT animal (name)>
<!ELEMENT name (#PCDATA)>
<!ATTLIST name lang CDATA #REQUIRED>
]>
<zoo>
  <animal>
    <name lang="en">Zebra</name>
  </animal>
  <animal>
    <name lang="en">Giraffe</name>
  </animal>
  <animal>
    <name lang="en">Cow</name>
  </animal>
</zoo>
```

### 4.3.2 Define as external DTD

Here's `myzoo.xml`:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE zoo SYSTEM zoo.dtd>
<zoo>
  <animal>
    <name lang="en">Zebra</name>
  </animal>
</zoo>
```

Which refers to `zoo.dtd` (this can be any URL):

```
<!ELEMENT zoo (animal)+>
<!ELEMENT animal (name)>
<!ELEMENT name (#PCDATA)>
<!ATTLIST name lang CDATA #REQUIRED>
```

#### 4.3.3 Define as reference, with additions

You can extend a referred external DTD:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE zoo SYSTEM zoo.dtd [
<!ENTITY favourite "Zebra">
]>
<zoo>
  <animal>
    <name lang="en">&favourite;</name>
  </animal>
</zoo>
```

#### 4.3.4 Define as reference to a PUBLIC DTD

No URL is given. It's up to the parser to "know" the public reference.

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN">
<html>

</html>
```

#### 4.3.5 Define as reference to a PUBLIC DTD, with URL

No URL is given. It's up to the parser to "know" the public reference.

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xht
<html xmlns="http://www.w3.org/1999/xhtml">

</html>
```

#### 4.3.6 Define as reference to a PUBLIC DTD, with URL and additions

Like before, we can make changes to the DTD.

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xht
<!ENTITY author "Jan">
]>
<html xmlns="http://www.w3.org/1999/xhtml">
  Written by &author;.
</html>
```

## 4.4 Contents of a DTD

### 4.4.1 Entities

Entities allow you to define shortcuts to refer to pieces of content.

1. Internal entity You can define an entity with the value in the same DTD:

   ```
   <!ENTITY home "Copenhagen">
   ```

   From there, you can refer to the entity anywhere in a document that's based on that DTD:

   ```
   <zoo>
     <description>A nice zoo in the center &home;</description>
   </zoo>
   ```

2. External entity You can also refer to an external URL for the contents of an entity.

   ```
   <!ENTITY home SYSTEM "home.txt">
   ```

   Or, you can include a public identifier if the entity is globally known to your systems (the URL is then optional, just like with DTD itself).

   ```
   <!ENTITY home PUBLIC "urn:mycorp:entities:home" "home.txt">
   ```

   Referring to the entity is unchanged:

   ```
   <zoo>
     <description>A nice zoo in the center of &home;</description>
   </zoo>
   ```

### 4.4.2 Elements

1. Elements: empty In case you want an element to contain neither child elements nor text (but might have attributes)

   ```
   <!ELEMENT important EMPTY>
   ```

   For example

   ```
   <important/>
   <important></important>
   ```

2. Elements: plain text `#PCDATA` is short for *Parsed character data*

   ```
   <!ELEMENT name (#PCDATA)>
   ```

   For example

   ```
   <name>Zebra</name>
   ```

3. Elements: child elements, sequence An element may have one, or several, child elements.

```
<!ELEMENT person (name, birthdate)>
```

For the example, each element in the sequence must occur exactly once, and in that order.

```
<person>
  <name>Jeff</name>
  <birthdate>1980-01-01</birthdate>
</person>
```

4. Elements: child elements, advanced The examples we've seen so far are instances of a more general description of element content:

   ```
   <!ELEMENT elmt (expr)>
   ```

   where `expr` is one of the following:

| Expression | Description |
|---|---|
| name | A single instance of a tag <name> |
| #PCDATA | Character data ( text body) |
| expr+ | One or more instances of the given expression |
| expr* | Zero or more instances of the given expression |
| expr? | Zero or one instances of the given expression |
| exprA, exprB | Expression exprA followed by exprB |
| exprA  exprB | Either expression exprA, OR expression exprB |
| (expr) | Any operators are applied to the expression as a whole |

   For example

   ```
   <!ELEMENT person (name+, job)>
   ```

5. Elements: unrestricted If you don't want to, or can't, describe the content of an element

   ```
   <!ELEMENT userdata ANY>
   ```

   For example

   ```
   <userdata>
     <favourite-zoo>Copenhagen</favourite-zoo>
   </userdata>
   ```

### 4.4.3 Attributes

1. Attributes: character data The attribute can contain any character data as value. A default value can be given.

   ```
   <!ATTLIST zoo
     location CDATA "Copenhagen"
   >
   ```

   Further attributes can be defined on subsequent lines (no other separator than whitespace).

   An example for the given DTD:

```
<zoo location="Odense"/>
```

2. Attributes: qualifiers

```
<!ATTLIST zoo
  location CDATA "Copenhagen">
```

   In the definition, in place of `"Copenhagen"`, one of the following qualifiers must be given:

   - `#IMPLIED` : The attribute is optional
   - `"value"` The attribute is optional (and `value` should be taken as default value)
   - `#REQUIRED` The attribute is required
   - `#FIXED "value"` The attribute is always assumed to be `value`. If it is given, it must be `value`.

3. Attributes: entity The attribute must be the name of any XML entity.

```
<!ATTLIST zoo
  location ENTITY #IMPLIED>
<!ENTITY home "Copenhagen">
```

   For example

```
<zoo location="home"/>
```

4. Attributes: entities The attribute must be the space-separated names of zero or more XML entities.

```
<!ATTLIST train
  stops ENTITY #IMPLIED>
<!ENTITY home "Copenhagen">
<!ENTITY work "Odense">
```

   For example

```
<train stops="home work"/>
```

5. Attributes: ID The attribute must be unique identifier (across the entire document and all tags)

```
<!ATTLIST zoo
   id ID #REQUIRED>
```

   For example

```
<zoo id="z1"/>
<zoo id="z2"/>
```

6. Attributes: ID reference The attribute must be the unique identifier of ANOTHER tag somewhere in the document.

```
<!ATTLIST zoo
  id ID #REQUIRED
  related IDREF #IMPLIED
>
```

For example

```
<zoo id="z1"/>
<zoo id="z2" related="z1"/>
```

7. Attributes: ID references The attribute must be the space-separated unique identifier(s) OTHER tag(s) somewhere in the document.

```
<!ATTLIST zoo
  id ID #REQUIRED
  related IDREFS #IMPLIED
>
```

For example

```
<zoo id="z1"/>
<zoo id="z3"/>
<zoo id="z2" related="z1 z3"/>
```

8. Attributes: token The attribute value must be a valid "token", i.e. characters or digits, and no whitespace.

```
<!ATTLIST zoo
  country NMTOKEN #IMPLIED
>
```

For example

```
<zoo country="da"/>
```

9. Attributes: tokens The attribute value must be a list of "tokens", i.e. characters or digits, and no whitespace.

```
<!ATTLIST zoo
  categories NMTOKENS #IMPLIED
>
```

For example

```
<zoo categories="pets reptiles"/>
```

10. Attributes: enumeration The attribute must be one of a fixed set of values (which themselves must be tokens).

```
<!ATTLIST tshirt
   size (xs | s | m | l | xl) #IMPLIED>
```

For example

```
<tshirt size="m"/>
```

#### 4.4.4 Parameter entities

Defining a DTD can repetitive. Hence, you can define entities that can be expanded *in the DTD itself*:

```
<!ENTITY % coolattrs
 "id          ID             #IMPLIED
  class       CDATA          #IMPLIED"
>
```

Now, when defining any element, we can add those attributes for free:

```
<!ATTLIST zoo
  %coolattrs;
  name CDATA #IMPLIED
>
```

### 4.5 Example DTD

Let's have a look at XHTML.

## 5 XPath demo session

<>

</>

## 6 Data modeling techniques

### 6.1 Challenges in data modeling

#### 6.1.1 Relationships

Imagine that in our zoo, an animal might have a preference to be with other animals.
How would you model this?

#### 6.1.2 Relationships

Imagine that in our zoo, an animal might have a preference to be with other animals.
How would you model this?

- As an attribute on `<animal>`?
- As a tag under `<animal>`?
- As several tags under `<animal>`?
- As something else entirely?

#### 6.1.3 Relationships

Imagine that in our zoo, an animal might have a preference to be with other animals.
How would you model this?

- As an attribute on `<animal>`?

- As a tag under `<animal>`?
- As several tags under `<animal>`?
- As something else entirely?

As with most things in IT, there is no one correct answer here.

#### 6.1.4 Expressing business rules

Imagine we are defining a flexible insurance system. Our rules are expressed in XML, so we can quickly change them.

- *When a person is younger than 24, they should pay DKK 200 extra for their car insurance*
- *When a household has gone a whole year with no home insurance claims, they get a DKK 100 discount on their home insurance*

How would you express this in XML?

#### 6.1.5 Code as data

We can go further than just business rules.

- Programming XML transformations: XSLT (more in XML Advanced course)
- Source code is a (abstract syntax) tree, so we can Convert it to XML

### 6.2 Versioning of data definitions

- Life cycle management
  - How long will we keep referring to this version of [widget]?
  - How long do we have to keep reading documents made using this version of [widget]?
  - We have a new idea of what [widget] should be, what do we do to out old documents?
- Communication
  - Find your stakeholders
- Professional approach to history, iterations, and versioning
  - Use a version control system like Git
  - Use semantic versioning for your schemas

### 6.3 Domain-Driven Design

- Focus on the various types of language used in business domains
- Use explicit scopes to designate a language boundary
- Use *event storming* workshops to discover process flows and entities (data)

### 6.4 XSD from code, or code from XSD

- Imagine the process through which understanding of your data came to be
- Was source code involved anywhere?
- Which would you pick (even if you're no developer):
  - Generate your XSD from source code
  - Generate your source code from XSD
  - Write your source code by hand, using XSD as a guide

### 6.5 Choosing a data format(s)

XML isn't the only way to exchange data

- XML (focus on semantics and precision)
- JSON (focus on simplicity)
- Protobuf (focus on binary compactness and backwards compatbility)

## 7 Welcome to the world of JSON

### 7.1 History

- 2001: JSON (loading Javascript dynamically)
- 2013: First "real" JSON standard from ECMA
- 2017: Latest current JSON standard, RFC 8259

Note: No semantic versioning

### 7.2 It's just Javascript, how hard can it be

- 2018:Test of about 50 JSON parsers

Quiz:

- How many of those JSON parsers handle all edge cases correctly?
- How many of those JSON parsers handle all edge cases in the same way?

### 7.3 Let's have schemas and types

- 2009...2022: *Draft 0* til *Draft 2020-12* of:
  - JSON schema
  - JSON Schema Validation

Types: `boolean`, `number` (and `"integer"`), `string`, `array`, `object`

- Where's date and time?

# 8 Demo XML: UBL invoice

## 8.1 A quick tour

Here's an XML Invoice example. Let's look at an UBL Invoice Creator .

## 8.2 Building blocks

- Core Component Types (CCTS, by UN/CEFACT)
- Unqualified Data Types (by OASIS)
- Common Basic Components (by UBL)
- Common Aggregate Components (by UBL)
- Invoice (by UBL)

# 9 Let's practice with git (live demo)

# 10 Specifications

- XML 1.0 (fifth edition)
- XML 1.1 (second edition)
- XSD 1.1 Part 1: Structure
- XSD 1.1 Part 2: Data types
- XPath 1.0
- XSLT 1.0
- WSDL 1.1 (SOAP)
- Whitespace

# 11 Online tools

- XSD Editor
- XML Validator (syntax)
- XML Validator (against XSD)

# 12 Windows tools

- MSYS2

# 13 Day 1 questionnaire

| | I am learning a lot | I know XML |
|---|---|---|
| Daniel | 5 | 2.5 |
| Anette | 2.5 | 2 |